IIS5008 Hardware Security
Final Project Report

Design of ECC Systems for Reliable Data Transmission

1. Names and Student IDs

Item	Information
Student ID	110070016
Name	蕭以勝

2. Overview of the ECC System Design

The implemented system follows the required encode-decode flow: the original message is encoded into a protected codeword, the codeword is corrupted by the Flash error model, and the decoder reconstructs the original message from the corrupted codeword.

The main design choice is an adaptive interleaved punctured convolutional code. A convolutional code is suitable for bit-level error correction because the Viterbi decoder can use sequence-level information instead of decoding each bit independently. Interleaving is added to reduce the impact of burst and clustered errors. Puncturing is used to control the redundancy ratio so that the encoded length remains within the required limit.

The system is adaptive because the aging parameter T is used to choose the protection strength. When T is small, the expected error severity is lower, so the encoder can use a lower redundancy ratio. When T is larger, the encoder increases the amount of protected information and approaches the maximum allowed redundancy ratio. This trade-off is important because the official score considers correctness, runtime, and redundancy.

```
m  -> encode(T) -> c  -> Flash Error Model -> c' -> decode(T) -> m'
```

3. Main Algorithms Used in the Design

3.1 Convolutional Encoding

The encoder uses a convolutional-code structure. For each input bit, the encoder updates a shift-register state and generates parity bits according to fixed generator masks. This creates redundancy across neighboring input bits, allowing the decoder to use the trellis structure to infer the most likely original bit sequence even when some received bits are corrupted.

3.2 Puncturing for Redundancy Control

A raw convolutional code may produce too many encoded bits, especially when termination bits are included. Therefore, the implementation applies deterministic puncturing. Puncturing removes selected parity observations while preserving a balanced distribution of the remaining observations. The target encoded length is selected so that the final codeword satisfies the project constraint:

```
|c_T| <= 2 |m|
```

This step is necessary because a design with excessive redundancy would be invalid even if the decoder is correct. In the public results, the redundancy ratios range from 1.5000 to 1.9625, which all satisfy the hard cap.

3.3 Interleaving

The encoded bits are permuted by a deterministic interleaver before being written to the output codeword. The purpose is to separate logically adjacent trellis observations in the physical codeword. As a result, a consecutive-byte burst or a localized disturb-like cluster becomes more distributed after deinterleaving. This makes the Viterbi decoder less likely to see one long, concentrated error region.

3.4 Viterbi Decoding

The decoder first maps the corrupted codeword back through the inverse interleaving process. It then applies Viterbi decoding on the convolutional-code trellis. For every trellis step, the decoder compares the observed parity information with the expected parity output for each candidate transition and keeps the path with the smallest accumulated metric. Finally, the best path is traced back to recover the original message bits.

4. Handling of the Flash Error Model and Aging Parameter T

The evaluation channel includes random bit errors, burst errors, retention-related degradation, and disturb-like clustered corruption. My design handles these errors through the combination of convolutional coding, interleaving, and T-adaptive redundancy.

Error Type	Challenge	Design Response
Random bit errors	Scattered bit flips may appear across the codeword.	The convolutional code and Viterbi decoder correct scattered bit-level corruption by selecting the most likely trellis path.
Burst errors	Several consecutive bytes may be corrupted in a local region.	Interleaving spreads consecutive physical errors across different logical trellis positions.
Retention errors	Error severity increases as the aging parameter T becomes larger.	The encoder increases redundancy for larger T to improve robustness under stronger aging-related degradation.
Disturb-like clustered errors	Corruption is concentrated in a small neighborhood rather than uniformly distributed.	The interleaver reduces the concentration of logical errors, and Viterbi decoding absorbs the remaining scattered errors.

4.1 Use of Aging Parameter T

The parameter T is treated as an indicator of storage-aging severity. The official specification describes that higher T corresponds to stronger retention-related corruption. Therefore, my implementation does not use one fixed code rate for all values of T. Instead, it selects a target redundancy level according to T. This allows the system to reduce overhead for easy cases while reserving stronger protection for difficult cases.

5. Experimental Results and Analysis

The following results were obtained by running the provided public test cases with the grading script. The implementation achieved 100.00% correctness on all public cases.

Testcase	Correctness	Runtime	Redundancy Ratio
public1	100.00%	612.8 ms	1.5000
public2	100.00%	3267.8 ms	1.6250
public3	100.00%	17528.7 ms	1.9500
public4	100.00%	50897.7 ms	1.9625

5.1 Correctness Analysis

All four public test cases reached 100.00% correctness. This shows that the design can recover the original message exactly under the provided public corruption patterns. Since correctness is the primary objective and is also used as a gate for runtime and redundancy scoring, maintaining perfect public correctness is important.

5.2 Redundancy Analysis

The redundancy ratio increases from public1 to public4. This is expected because more difficult cases require stronger protection. The ratios for public3 and public4 are close to 2.0, which means the design uses nearly the maximum allowed redundancy to improve reliability. However, the ratios remain below the hard cap, so the encoded output is valid.

5.3 Runtime Analysis

The runtime increases significantly from public1 to public4. This is mainly caused by the larger number of Viterbi decoding steps and the stronger protection used in difficult cases. Nevertheless, the implementation includes several optimizations to reduce unnecessary bit-vector conversion, memory copying, and repeated distance computation.

5.4 Trade-off Discussion

The results show a clear trade-off. Lower redundancy reduces storage overhead and runtime, but may not provide enough protection for severe corruption. Higher redundancy improves robustness but increases both encoded length and decoding cost. My design prioritizes correctness first, then reduces overhead and runtime where possible. This strategy is reasonable because the scoring rule gives correctness the highest importance and applies a correctness gate before considering runtime and redundancy.

6. Important Implementation Details

The implementation was written in C++ and avoids external libraries. Several low-level optimizations were applied in the newer version to improve runtime without changing the decoding behavior.

Direct bit access: The optimized version uses get_bit() and set_bit() to directly read and write individual bits inside byte arrays. This avoids expanding every byte into a separate temporary bit vector.

Integrated interleaving: Instead of generating a full logical encoded bit array and then performing a separate interleaving pass, the encoder computes the physical interleaved position when writing each kept bit.

Interleaver plan: The interleaver parameters are computed once and reused. This avoids repeatedly recomputing stride, offset, mask, or bit-reversal information.

Distance lookup table: Repeated Viterbi branch-metric computations are replaced by a small lookup table. This reduces the cost of the inner decoding loop.

Compact Viterbi storage: The decoder stores only the necessary traceback choice information, reducing memory usage and improving cache locality.

```
High-level optimized flow:
input bytes -> direct bit access -> convolutional encoding + puncturing ->
interleaved codeword
corrupted codeword -> deinterleaving-aware Viterbi decoding -> recovered message
bytes
```

7. What I Learned and Problems Encountered

- I learned that ECC design is not only about choosing a stronger code. It also requires balancing reliability, redundancy, and runtime.
- A simple random-error assumption is not enough for Flash-like channels. Burst and clustered errors require additional techniques such as interleaving.
- The hard encoded-length cap makes the design more challenging. Even if a code is strong, it cannot be used if the redundancy exceeds the limit.
- Runtime optimization matters because Viterbi decoding is repeatedly executed for many values of T. Small improvements in the inner loop can produce visible runtime gains.

8. Suggestions and Feedback

This project was useful because it connected ECC theory with realistic hardware-inspired error patterns. The use of random, burst, retention, and disturb-like errors made the problem more practical than a simple independent bit-flip channel.

One possible improvement is to provide an optional visualization or small example of how the Flash error model corrupts the codeword. Another useful addition would be an extra public testcase that focuses mainly on burst or clustered errors, so students can more directly evaluate the benefit of interleaving. Overall, the project helped me understand how practical ECC systems must consider correctness, redundancy overhead, and computational efficiency at the same time.